REPUBLIQUE DU CAMEROUN
****
PAIX-TRAVAL-PATRIE
****
UNIVERSITE DE DSCHAN
****
FACULTES DES SCIENCES
****
DEPARTEMENT DE SCIENCES

REPUBLIC OF CAMEROON
****
PEACE-WORK-FATHERLAND
****
UNIVERSITY OF DSCHANG
****
FACULTY OF SCIENCES
****
DEPARTEMENT OF SCIENCES

IAR418 - Fouille de Données

CHAPITRE I : Introduction au Data Mining

Présenté par :

DNJOMOU YONMBA WILFRIED LOIC	**CM-UDS-24SCI0999**
KENFACK LEONEL	**CM-UDS-17SCI0998**
SAGUEU WAKAM DILANE	**CM-UDS-24SCI1040**

Superviseur : Pr. KENGNE TCHENDJI Vianney

Année académique 2024-2025

Table des matières

Introduction générale **2**

1 Combinaison de classifieurs **3**

1.1 Définition et phases de la combinaison de classifieurs 3

1.1.1 Nature des sorties et niveaux d'information 4

1.2 Motivations théoriques et justifications statistiques 4

1.2.1 Théorème du jury de Condorcet 5

1.2.2 Décomposition biais–variance 5

1.2.3 Théorie unifiée de la diversité dans les ensembles 5

1.2.4 Les problèmes de Dietterich 7

1.3 Achirtectures des systèmes multi-classifieurs 7

1.3.1 Architectures de contrôle de flux 7

1.3.2 Systèmes de sélection dynamique (DS) 8

1.4 Typologies des méthodes d'ensemble 9

1.4.1 Bagging (bootstrap aggregating) 9

1.4.2 Boosting 12

1.4.3 Stacking (stacked generalization) 15

1.4.4 Voting (vote majoritaire vs vote probabiliste) 19

1.5 Analyse comparative des approches 19

1.6 Avantages et limites de la combinaison 20

1.6.1 Avantages 20

1.6.2 Limites 20

Conclusion générale **21**

Introduction générale

L'intelligence artificielle et l'apprentissage automatique occupent une place croissante dans les systèmes de décision, industriels, médicaux, financiers ou réglementaires. La **combinaison de classifieurs**, ou apprentissage d'ensemble (*ensemble learning*), s'impose comme un paradigme central : on construit un comité de modèles dont les prédictions sont agrégées pour améliorer la précision et la robustesse (bagging, boosting, stacking). Parallèlement, l'**explicabilité des modèles** est devenue une exigence majeure : les décisions automatisées sont scrutées par les utilisateurs, les régulateurs et la société ; comprendre *pourquoi* un modèle prédit relève de la confiance, de la conformité et de l'éthique. Les méthodes d'explicabilité (intrinsèques ou post-hoc : LIME, SHAP, importance des variables) visent à rendre les systèmes moins opaques. Ce rapport se situe à l'intersection de ces deux thématiques : les systèmes d'ensemble complexifient le modèle global et renforcent l'effet « boîte noire », tout en offrant des gains de performance substantiels.

Les méthodes d'ensemble améliorent la précision au prix d'une complexité accrue (plusieurs modèles, fusion, dépendances), ce qui soulève une question centrale : *comment concilier la puissance des systèmes combinant plusieurs classifieurs avec l'exigence d'explicabilité et d'interprétabilité des décisions ?* D'un côté, un classifieur unique peut être plus facile à expliquer ; de l'autre, un ensemble peut être plus performant mais plus difficile à interpréter. La problématique recouvre les dimensions techniques, architecturales et réglementaires (transparence, traçabilité).

Les objectifs sont de présenter les fondements de la combinaison de classifieurs (définitions, typologies, motivations théoriques, aspects architecturaux) ; de clarifier les concepts d'explicabilité et d'interprétabilité et les principales méthodes (intrinsèques et post-hoc : LIME, SHAP, PDP/ICE, explicabilité locale et globale) ; d'analyser les enjeux spécifiques aux ensembles (complexité, stratégies d'explication, compromis performance et compréhension) ; et de mettre en perspective les avantages et limites au regard de l'explicabilité.

Le document comporte trois parties : la **combinaison de classifieurs** (définition, typologies, aspects architecturaux, analyse comparative, avantages et limites) ; l'**explicabilité des modèles** (définition, interprétabilité, méthodes, enjeux dans les systèmes décisionnels, éthique et réglementation) ; la **combinaison et l'explicabilité** (boîte noire, stratégies, compromis performance et compréhension). La conclusion générale synthétise les apports et ouvre sur les perspectives.

Chapitre 1

Combinaison de classifieurs

Introduction

L'évolution de l'apprentissage automatique a franchi une étape décisive avec l'émergence des systèmes de classifieurs multiples (*Multiple Classifier Systems – MCS*), plus communément désignés sous le terme d'apprentissage d'ensemble [11, 33, 18]. Cette approche repose sur un paradigme fondamental : la combinaison stratégique de plusieurs modèles de prédiction pour pallier les insuffisances inhérentes à tout classifieur unique.

Historiquement, la recherche en intelligence artificielle s'est concentrée sur l'optimisation d'algorithmes individuels. Cependant, l'expérience a démontré qu'un modèle unique, aussi sophistiqué soit-il, est souvent limité par ses propres biais structurels et sa sensibilité au bruit des données. L'apprentissage d'ensemble propose de substituer au modèle solitaire un comité de décideurs dont la sagesse collective surpasse les capacités individuelles.

Cette section propose une exploration de la combinaison de classifieurs, structurée autour des fondements théoriques, des typologies méthodologiques et des considérations architecturales qui définissent ce domaine. L'analyse s'attache à montrer comment la diversité des modèles et la rigueur de leur agrégation permettent d'atteindre des niveaux de performance et de robustesse inaccessibles aux approches classiques.

1.1 Définition et phases de la combinaison de classifieurs

La **combinaison de classifieurs** est définie comme le processus d'intégration des sorties de plusieurs modèles d'apprentissage afin de générer une décision globale plus fiable et précise. Un système multi-classifieurs est structurellement composé d'un ensemble d'apprenants de base, souvent qualifiés d'*apprenants faibles* (*weak learners*) lorsqu'ils sont pris isolément, et d'un mécanisme de fusion qui orchestre leurs réponses.

Le cycle de vie d'un tel système se décompose généralement en trois phases critiques :

1. **Génération de l'ensemble** : cette phase initiale consiste à produire un pool de modèles de base. L'objectif est de créer des modèles qui ne sont pas seulement performants, mais surtout complémentaires. La complémentarité est assurée par l'introduction de diversité, que ce soit par l'utilisation de différents sous-ensembles de données, de paramètres variés pour un même algorithme, ou de familles d'algorithmes distinctes.

2. **Sélection ou élagage (*pruning*)** : une fois le pool généré, il peut être nécessaire de réduire la taille de l'ensemble. L'élagage vise à éliminer les modèles redondants ou ceux dont la performance dégrade la décision collective. Cette étape permet non seulement d'améliorer la précision globale mais aussi de réduire les coûts computationnels lors de l'inférence.
3. **Combinaison ou fusion** : la phase finale consiste à fusionner les prédictions des modèles sélectionnés pour obtenir la réponse ultime. Le choix de la fonction de combinaison dépend crucialement du type d'information fournie par les classifieurs de base (niveau d'information en sortie).

1.1.1 Nature des sorties et niveaux d'information

L'efficacité de la combinaison dépend de la richesse des informations transmises par les modèles individuels. On distingue classiquement trois niveaux de sortie :

— **Niveau « Classe » (hard output)** : le classifieur fournit uniquement l'étiquette de la classe prédite. C'est l'information la plus élémentaire, où chaque modèle agit comme un votant binaire.

— **Niveau « Rang »** : le classifieur produit une liste ordonnée des classes possibles, de la plus probable à la moins probable. Ce format est particulièrement utile lorsque la classe réelle ne figure pas au premier rang mais reste parmi les premières propositions.

— **Niveau « Mesure » (soft output)** : c'est le niveau le plus riche. Le classifieur associe à chaque classe une valeur numérique, telle qu'une probabilité ou un score de confiance. Cette mesure quantifie l'incertitude du modèle et permet des fusions pondérées beaucoup plus fines.

Le tableau suivant présente une comparaison structurelle de ces types d'informations en sortie.

TABLE 1.1 – Comparaison des niveaux d'information en sortie des classifieurs

Type de sortie	Information contenue	Complexité de fusion	Cas d'utilisation typique
Classe	Étiquette discrète uniquement	Faible	Vote majoritaire simple
Rang	Préférences relatives ordonnées	Moyenne	Méthode de Borda, systèmes de recommandation
Mesure	Probabilités, scores de confiance	Élevée	Moyenne pondérée, stacking, modèles bayésiens

1.2 Motivations théoriques et justifications statistiques

La supériorité des systèmes multi-classifieurs n'est pas une simple observation empirique ; elle s'appuie sur des piliers théoriques robustes, notamment le théorème du jury de Condorcet et la décomposition biais–variance [30, 1, 14].

1.2.1 Théorème du jury de Condorcet

Formulé à l'origine dans le contexte des sciences sociales, ce théorème constitue une base théorique fondamentale pour l'apprentissage d'ensemble [34, 7]. Il postule que si un groupe d'individus doit prendre une décision binaire (par exemple, classer un objet comme A ou B) et que chaque individu a une probabilité p supérieure à 0,5 de choisir la bonne réponse, alors la probabilité que la décision de la majorité soit correcte augmente avec le nombre d'individus.

Mathématiquement, si L est le nombre de classifieurs indépendants et p leur précision individuelle, la probabilité P que l'ensemble prenne la bonne décision par vote majoritaire tend vers 1 lorsque L tend vers l'infini. À l'inverse, si $p < 0{,}5$, l'ajout de classifieurs réduit la performance globale. Cette démonstration justifie l'utilisation d'« apprenants faibles » pour construire des systèmes puissants, à condition qu'ils soient plus performants qu'un tirage aléatoire et qu'ils soient suffisamment indépendants.

1.2.2 Décomposition biais–variance

D'un point de vue statistique, l'erreur de généralisation d'un modèle d'apprentissage peut être décomposée en trois composantes : le biais au carré, la variance et l'erreur irréductible (bruit) [32, 16]. La formule de l'erreur quadratique moyenne (MSE) est :

$$\mathbb{E}\big[(y - \hat{f}(x))^2\big] = \mathrm{Bias}\big[\hat{f}(x)\big]^2 + \mathrm{Var}\big[\hat{f}(x)\big] + \sigma^2, \tag{1.1}$$

où σ^2 représente le bruit inhérent aux données.

— Le **biais** mesure l'écart entre la prédiction moyenne du modèle et la valeur réelle. Un biais élevé est le signe d'un modèle trop simpliste qui ne capture pas la complexité des données (sous-apprentissage, *underfitting*).

— La **variance** mesure la dispersion des prédictions pour différents jeux de données d'apprentissage. Une variance élevée indique que le modèle est trop sensible aux fluctuations et au bruit des données (sur-apprentissage, *overfitting*).

L'apprentissage d'ensemble permet de naviguer dans ce compromis biais–variance. Les méthodes de type bagging agissent principalement en réduisant la variance par la moyenne de modèles complexes, tandis que les méthodes de boosting se concentrent sur la réduction du biais en agrégeant itérativement des modèles simples pour corriger leurs erreurs systématiques.

1.2.3 Théorie unifiée de la diversité dans les ensembles

Une contribution théorique importante aux méthodes d'ensemble consiste à faire apparaître explicitement la diversité comme composante de la décomposition de la perte. Pour une vaste famille de fonctions de perte (notamment les divergences de Bregman, qui incluent l'erreur quadratique moyenne et l'entropie croisée), on peut montrer que la perte de l'ensemble s'exprime comme la moyenne des pertes individuelles diminuée d'un terme de diversité [24].

Dans le cas particulier de l'erreur quadratique moyenne, cette idée est capturée par l'identité de Krogh et Vedelsby [23]. Pour un ensemble de M modèles dont les prédictions sont

$q_1(x), \ldots, q_M(x)$ pour une entrée x et une cible y, on pose

$$\bar{q}(x) = \frac{1}{M} \sum_{i=1}^{M} q_i(x).$$

L'identité s'écrit alors :

$$\big(\bar{q}(x) - y\big)^2 = \frac{1}{M} \sum_{i=1}^{M} \big(q_i(x) - y\big)^2 - \frac{1}{M} \sum_{i=1}^{M} \big(q_i(x) - \bar{q}(x)\big)^2. \tag{1.2}$$

Le membre de gauche est l'erreur quadratique de l'ensemble, le premier terme à droite la moyenne des erreurs quadratiques individuelles, et le second terme (souvent appelé « ambiguïté » ou « diversité ») mesure le degré de désaccord entre les prédictions des modèles. L'identité montre que, toutes choses égales par ailleurs, l'erreur de l'ensemble diminue lorsque la diversité augmente.

Démonstration de l'identité de Krogh et Vedelsby. Pour simplifier les notations, on note $\bar{q} = \bar{q}(x)$ et $q_i = q_i(x)$.

(i) Développement de l'erreur de l'ensemble.

$$(\bar{q} - y)^2 = \bar{q}^2 - 2\bar{q}y + y^2.$$

(ii) Développement de la moyenne des erreurs individuelles.

$$\begin{aligned}
\frac{1}{M} \sum_{i=1}^{M} (q_i - y)^2 &= \frac{1}{M} \sum_{i=1}^{M} (q_i^2 - 2q_i y + y^2) && (1.3)\\
&= \frac{1}{M} \sum_{i=1}^{M} q_i^2 - 2y \left(\frac{1}{M} \sum_{i=1}^{M} q_i \right) + y^2 && (1.4)\\
&= \frac{1}{M} \sum_{i=1}^{M} q_i^2 - 2y\bar{q} + y^2. && (1.5)
\end{aligned}$$

(iii) Développement du terme de diversité.

$$\begin{aligned}
\frac{1}{M} \sum_{i=1}^{M} (q_i - \bar{q})^2 &= \frac{1}{M} \sum_{i=1}^{M} (q_i^2 - 2q_i\bar{q} + \bar{q}^2) && (1.6)\\
&= \frac{1}{M} \sum_{i=1}^{M} q_i^2 - 2\bar{q} \left(\frac{1}{M} \sum_{i=1}^{M} q_i \right) + \bar{q}^2 && (1.7)\\
&= \frac{1}{M} \sum_{i=1}^{M} q_i^2 - 2\bar{q}^2 + \bar{q}^2 && (1.8)\\
&= \frac{1}{M} \sum_{i=1}^{M} q_i^2 - \bar{q}^2. && (1.9)
\end{aligned}$$

(iv) Soustraction des deux termes.

$$\frac{1}{M}\sum_{i=1}^{M}(q_i - y)^2 - \frac{1}{M}\sum_{i=1}^{M}(q_i - \bar{q})^2 = \left(\frac{1}{M}\sum_{i=1}^{M} q_i^2 - 2y\bar{q} + y^2\right) - \left(\frac{1}{M}\sum_{i=1}^{M} q_i^2 - \bar{q}^2\right) \tag{1.10}$$

$$= -2y\bar{q} + y^2 + \bar{q}^2 \tag{1.11}$$

$$= (\bar{q} - y)^2. \tag{1.12}$$

On retrouve bien l'identité annoncée. Comme le terme de diversité est une somme de carrés, il est toujours positif ou nul. L'erreur de l'ensemble est donc toujours inférieure ou égale à la moyenne des erreurs individuelles, avec égalité lorsque tous les modèles prédisent la même valeur (diversité nulle).

Cette identité justifie l'importance de promouvoir la diversité dans les ensembles (via le bagging, le boosting, la randomisation des architectures, etc.) et ouvre sur des généralisations à d'autres pertes au moyen des divergences de Bregman [24].

1.2.4 Les problèmes de Dietterich

Thomas Dietterich a identifié trois raisons fondamentales expliquant pourquoi un ensemble peut surpasser un modèle unique :

1. **Le problème statistique** : lorsque le volume de données est faible par rapport à la taille de l'espace des hypothèses, l'algorithme peut trouver plusieurs modèles qui s'ajustent parfaitement aux données d'entraînement. En les combinant, l'ensemble réduit le risque de choisir par erreur un modèle qui généralise mal.
2. **Le problème computationnel** : de nombreux algorithmes (comme les réseaux de neurones) utilisent des heuristiques de recherche locale qui peuvent rester bloquées dans des optima locaux. Un ensemble effectuant plusieurs recherches à partir de points de départ différents a plus de chances de trouver une meilleure approximation globale.
3. **Le problème de représentation** : parfois, la véritable fonction cible ne peut être représentée par aucun des modèles de l'espace d'hypothèses choisi. La combinaison de plusieurs modèles permet d'étendre cet espace et d'approcher des fonctions plus complexes.

1.3 Achirtectures des systèmes multi-classifieurs

L'architecture d'un système multi-classifieurs définit la manière dont les informations circulent entre les composants et la façon dont les ressources sont exploitées.

1.3.1 Architectures de contrôle de flux

On distingue trois organisations principales :

1. **Architecture parallèle** : tous les classifieurs opèrent indépendamment et simultanément. La fusion intervient à la fin. Cette structure est neutre (aucun classifieur n'influence les autres) et hautement parallélisable sur du matériel multi-cœurs.

2. **Architecture séquentielle (en série)** : les classifieurs sont organisés en cascade. Un premier classifieur rapide effectue une prédiction ; s'il échoue ou exprime une faible confiance, l'exemple est passé à un second modèle plus complexe. Cela permet d'économiser des ressources de calcul en ne sollicitant les modèles coûteux que pour les cas difficiles.
3. **Architecture hybride** : elle combine les deux approches. Par exemple, un premier étage parallèle peut générer un consensus, dont le résultat est validé par un classifieur de contrôle séquentiel.

Ces formes peuvent être illustrées par des figures dédiées aux architectures parallèle et séquentielle.

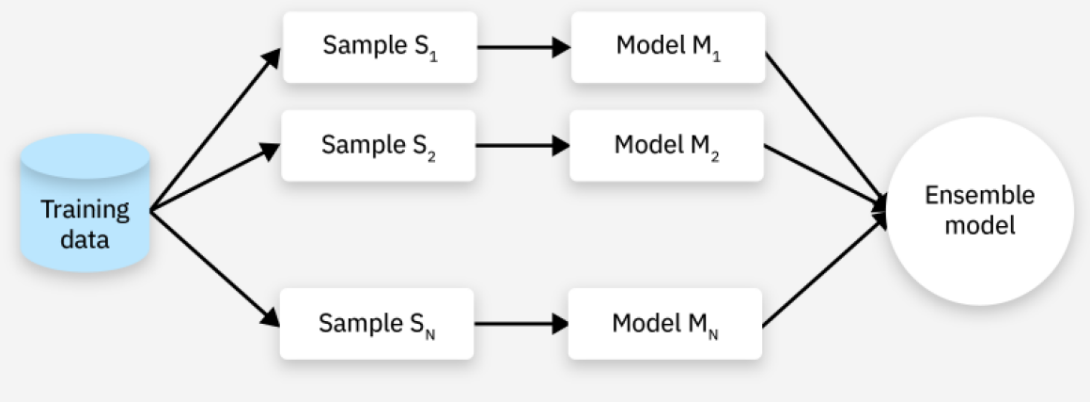

FIGURE 1.1 – Architecture parallèle pour un système multi-classifieurs.

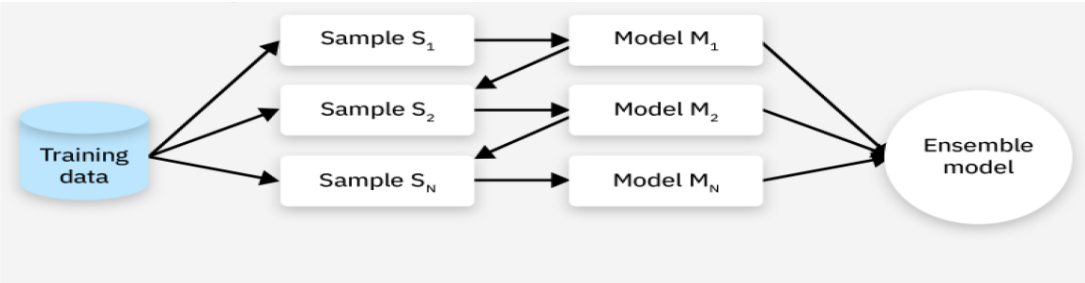

FIGURE 1.2 – Architecture séquentielle pour un système multi-classifieurs.

1.3.2 Systèmes de sélection dynamique (DS)

Une tendance majeure consiste à ne pas utiliser tous les classifieurs pour chaque exemple, mais à sélectionner « à la volée » le ou les modèles les plus compétents [9, 28, 22, 8, 10].

— **Logique de compétence locale** : on définit une « région de compétence » autour d'un exemple test (en utilisant par exemple ses plus proches voisins dans le jeu d'entraînement). On évalue ensuite quels modèles du pool ont été les plus performants dans cette zone spécifique.

— **DCS (dynamic classifier selection)** : on choisit l'unique meilleur modèle pour l'exemple donné.

— **DES (dynamic ensemble selection)** : on construit un sous-ensemble de modèles experts pour cette région particulière.

1.4 Typologies des méthodes d'ensemble

Les méthodes d'ensemble se divisent principalement en approches parallèles (bagging), séquentielles (boosting) et hiérarchiques (stacking) [4, 36, 26].

1.4.1 Bagging (bootstrap aggregating)

Le bagging (pour *bootstrap aggregating*) vise à réduire la variance en entraînant plusieurs modèles indépendants en parallèle sur des échantillons de données légèrement différents. Introduit par Breiman [2] et largement diffusé ensuite [19, 17, 1], il est particulièrement efficace pour des apprenants de base instables comme les arbres de décision profonds.

Définition. À partir d'un ensemble d'apprentissage $D = \{(x_j, y_j)\}_{j=1}^n$, le bagging construit m échantillons bootstrap D_i par tirage aléatoire avec remise, chacun de taille n. Sur chaque D_i, on entraîne un modèle de base $\hat{f}_i$ (*base learner*). À la prédiction, les sorties des m modèles sont agrégées :

— en régression, par moyenne

$$\hat{f}(x) = \frac{1}{m} \sum_{i=1}^{m} \hat{f}_i(x);$$

— en classification, par vote majoritaire

$$\hat{y}(x) = \arg\max_k \sum_{i=1}^{m} \mathbb{I}\big(\hat{y}_i(x) = k\big).$$

La construction des m modèles étant indépendante, l'entraînement est naturellement parallélisable et s'adapte bien aux architectures multi-cœurs ou distribuées [4, 19].

Fonctionnement détaillé. On peut résumer la procédure de bagging en quatre étapes [2, 25] :

1. **Choix de l'apprenant de base et du nombre de modèles** : on fixe un modèle de base (par exemple, un arbre de décision) et un nombre de réplications m (souvent entre 50 et 500).
2. **Génération des échantillons bootstrap** : pour chaque $i \in \{1, \ldots, m\}$, on tire n exemples de D avec remise pour former D_i. Chaque observation a une probabilité $1/n$ d'être choisie à chaque tirage ; au total, environ 63,2 % des exemples distincts apparaissent dans D_i, les autres constituant les données *out-of-bag*.
3. **Entraînement des modèles** : on entraîne indépendamment un modèle $\hat{f}_i$ sur chaque D_i. Cette étape est triviale à paralléliser.
4. **Agrégation des prédictions** : pour une nouvelle entrée x, on calcule les prédictions des m modèles et on les agrège par moyenne (régression) ou vote (classification), comme indiqué ci-dessus.

Un pseudocode simplifié est le suivant :

```
fonction Bagging(D, m, base_learner):
    modèles = []
    pour i de 1 à m:
        D_i = bootstrap_sample(D)      // tirage avec remise
        f_i = base_learner.fit(D_i)
        modèles.ajouter(f_i)
    retour modèles

fonction Predire(modèles, x):
    prédictions = [f_i.predict(x) pour f_i dans modèles]
    si régression:      retour moyenne(prédictions)
    si classification:  retour mode(prédictions)
```

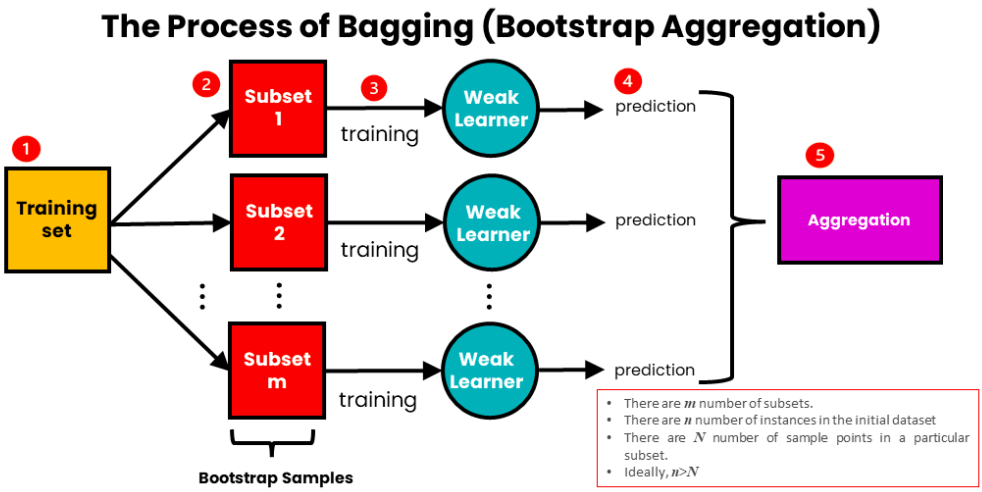

FIGURE 1.3 – Schéma pédagogique du bagging : du jeu de données original aux échantillons bootstrap, puis aux modèles de base et à l'agrégation.

Rôle des données out-of-bag (OOB). Lors de la génération de D_i, chaque exemple de D a une probabilité $(1 - 1/n)^n \simeq 1/e$ de ne jamais être sélectionné ; environ 36,8 % des observations restent donc en dehors de chaque bootstrap et sont dites *out-of-bag* pour le modèle $\hat{f}_i$ considéré.

Pour une observation (x_j, y_j), on note

$$S_j = \{\, i \in \{1, \ldots, m\} \mid (x_j, y_j) \notin D_i \,\}$$

l'ensemble des indices de modèles qui n'ont pas vu ce point à l'entraînement. On peut alors définir une prédiction OOB par

$$\tilde{f}_j(x_j) = \frac{1}{|S_j|} \sum_{i \in S_j} \hat{f}_i(x_j)$$

en régression, ou par vote majoritaire restreint aux $i \in S_j$ en classification. Une estimation de l'erreur OOB est donnée par

$$\varepsilon_{\mathrm{OOB}} = \frac{1}{n} \sum_{j=1}^{n} \ell\big(\tilde{f}_j(x_j), y_j\big),$$

où ℓ est la fonction de perte (MSE, entropie croisée, etc.). Cette quantité fournit une approximation fiable de l'erreur de généralisation, sans avoir besoin de réserver un jeu de validation distinct [2, 25, 19].

Réduction de la variance. Le principal intérêt du bagging est de réduire la variance des prédictions. Dans un scénario idéal où les modèles $\hat{f}_i$ seraient indépendants et de variance v, la variance de la moyenne

$$\hat{f}(x) = \frac{1}{m} \sum_{i=1}^{m} \hat{f}_i(x)$$

serait v/m. En pratique, les modèles ne sont pas indépendants ; si l'on note ρ la corrélation moyenne entre deux prédicteurs, on obtient approximativement [7] :

$$\mathrm{Var}\big[\hat{f}(x)\big] \approx \rho v + \frac{1-\rho}{m} v.$$

Quand m augmente, le second terme décroît comme $1/m$ et la variance tend vers ρv. Le bagging est donc particulièrement pertinent lorsque les modèles de base sont peu biaisés mais très variables (comme les arbres profonds) : il stabilise leurs prédictions sans augmenter significativement le biais.

Exemples d'algorithmes baggés. *Forêt aléatoire.* La forêt aléatoire (*Random Forest*) est l'extension la plus célèbre du bagging. En plus du bootstrapping, elle introduit une randomisation des caractéristiques (*features*) : à chaque division d'un nœud de l'arbre, seul un sous-ensemble aléatoire de variables est considéré. Cela décorrèle les arbres et renforce encore la réduction de variance [2, 19].

Un aperçu schématique de l'algorithme Random Forest est donné par la figure 1.4.

Exemple : bagging avec k plus proches voisins. Le bagging ne se limite pas aux arbres. Par exemple, appliqué au classifieur des k plus proches voisins (k-NN), il permet de stabiliser un modèle naturellement très sensible aux fluctuations locales des données. On génère m jeux bootstrap D_i, chacun servant de base à un modèle k-NN (qui consiste essentiellement à mémoriser D_i). Pour une nouvelle observation x, on agrège les m prédictions k-NN par vote ou moyenne des probabilités estimées [14, 4].

Dans les bibliothèques modernes comme scikit-learn, ces variantes sont disponibles via la classe `BaggingClassifier` en choisissant simplement l'estimateur de base (arbre, k-NN, etc.) et en activant le calcul de la « OOB score » pour obtenir automatiquement une estimation interne de l'erreur de généralisation.

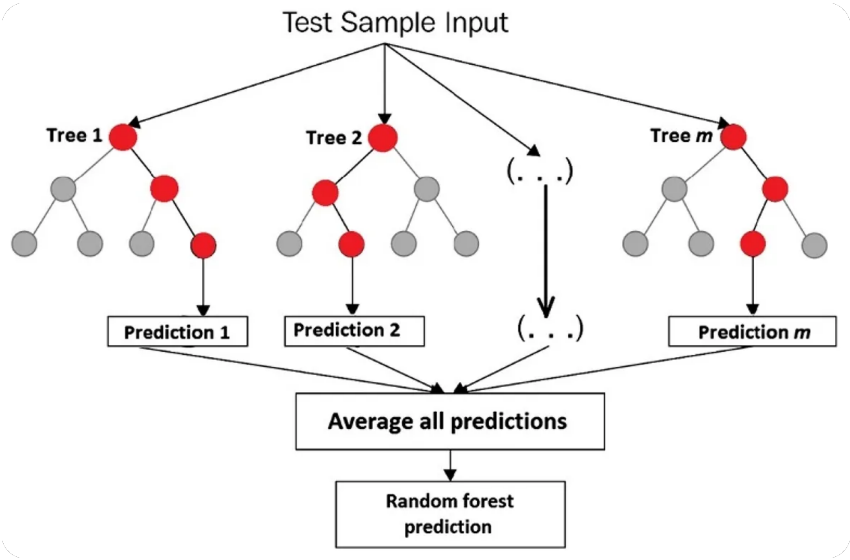

FIGURE 1.4 – Aperçu de l'algorithme Random Forest.

1.4.2 Boosting

Le boosting regroupe une famille de méthodes séquentielles où les modèles sont entraînés les uns après les autres, chacun se concentrant sur les erreurs commises par les précédents. Contrairement au bagging qui agit surtout sur la variance, le boosting cherche principalement à réduire le biais, en construisant progressivement un modèle additif puissant à partir de modèles de base faibles [12, 27, 4, 17].

Principe général. On considère un ensemble d'entraînement $\{(x_i, y_i)\}_{i=1}^n$ avec $y_i \in \{-1, +1\}$ pour fixer les idées. Le boosting maintient une distribution de poids $D_t(i)$ sur les exemples. À chaque itération t :

1. un *apprenant faible* h_t est entraîné sur les données pondérées par D_t ;
2. son erreur pondérée $\varepsilon_t = \sum_i D_t(i)\, \mathbb{I}\big(h_t(x_i) \neq y_i\big)$ est évaluée ;
3. un coefficient α_t mesurant la confiance accordée à h_t est calculé ;
4. les poids D_t sont mis à jour pour donner plus d'importance aux exemples mal prédits.

Le modèle final est une combinaison pondérée des h_t. Sous l'hypothèse de « faible apprentissage » (chaque h_t fait un peu mieux que le hasard), on peut montrer que l'erreur d'entraînement décroît de manière exponentielle avec le nombre d'itérations [12, 27].

AdaBoost : formulation et réduction du biais. AdaBoost (Adaptive Boosting) est l'algorithme de boosting le plus classique [12, 27]. Sa version binaire peut se décrire comme suit.

1. **Initialisation.** On part d'une distribution uniforme

$$D_1(i) = \frac{1}{n}, \quad i = 1, \ldots, n.$$

2. **Pour** $t = 1, \ldots, T$ **:**

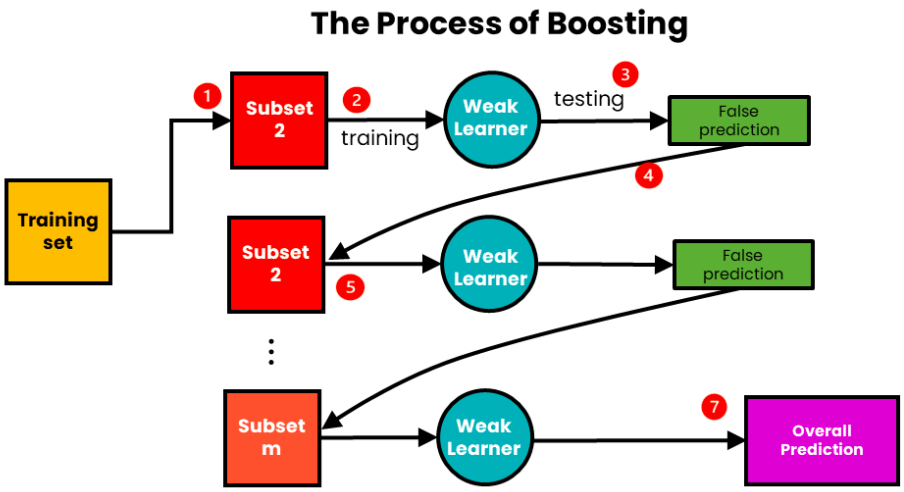

FIGURE 1.5 – Schéma pédagogique du boosting : chaîne de modèles où chaque étape corrige les erreurs de la précédente.

(a) on entraîne un modèle faible $h_t : \mathcal{X} \to \{-1, +1\}$ en minimisant l'erreur pondérée

$$\varepsilon_t = \sum_{i=1}^{n} D_t(i)\, \mathbb{I}\big(h_t(x_i) \neq y_i\big),$$

avec la contrainte $\varepsilon_t < 1/2$;

(b) on définit le poids du modèle

$$\alpha_t = \frac{1}{2} \log\left(\frac{1 - \varepsilon_t}{\varepsilon_t}\right) ;$$

(c) on met à jour les poids des exemples :

$$D_{t+1}(i) = \frac{D_t(i)\, \exp\big(-\alpha_t y_i h_t(x_i)\big)}{Z_t},$$

où Z_t est un facteur de normalisation assurant $\sum_i D_{t+1}(i) = 1$. Les exemples mal classés ($y_i h_t(x_i) < 0$) voient leur poids augmenter, ce qui force h_{t+1} à se focaliser sur eux.

Le classifieur final est

$$H(x) = \operatorname{sign}\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right).$$

Théoriquement, on peut montrer que l'erreur d'entraînement de H est majorée par [27, 12]

$$\mathrm{err}_{\mathrm{train}}(H) \leq \prod_{t=1}^{T} Z_t \leq \exp\left(-2 \sum_{t=1}^{T} \gamma_t^2\right), \quad \text{où } \gamma_t = \tfrac{1}{2} - \varepsilon_t.$$

Si chaque apprenant faible est au moins un peu meilleur que le hasard ($\gamma_t \geq \gamma > 0$), l'erreur d'entraînement décroît donc comme $\exp(-2\gamma^2 T)$. En pratique, cela signifie que le biais du modèle global diminue rapidement avec le nombre d'itérations, tant que les apprenants restent

effectivement faibles et diversifiés.

On peut aussi interpréter AdaBoost comme une méthode qui minimise de façon gloutonne la perte exponentielle

$$L(f) = \sum_{i=1}^{n} \exp\bigl(-y_i f(x_i)\bigr), \quad f(x) = \sum_{t=1}^{T} \alpha_t h_t(x),$$

par une descente de gradient coordonnée dans l'espace des fonctions. Cette vision explique pourquoi la marge $y_i f(x_i)$ joue un rôle central dans la performance [27].

En pratique, AdaBoost est souvent implémenté avec des *decision stumps* (arbres de profondeur 1) comme apprenants faibles ; dans scikit-learn, on peut l'utiliser via la classe `AdaBoostClassifier` avec un `DecisionTreeClassifier(max_depth=1)` comme estimateur de base. La complexité est en gros proportionnelle à T fois le coût d'entraînement d'un apprenant faible, et l'algorithme est naturellement itératif.

Gradient boosting : point de vue en espace de fonctions. Le gradient boosting généralise l'idée du boosting à toute fonction de perte différentiable [13]. On cherche à approximer une fonction $F^\star$ en construisant un modèle additif

$$F_T(x) = \sum_{t=1}^{T} \nu\, \beta_t\, h_t(x),$$

où h_t sont des apprenants faibles (souvent des arbres de décision), β_t des coefficients et $\nu \in (0, 1]$ un facteur de « shrinkage » (taux d'apprentissage).

À chaque itération t :

1. on dispose d'un modèle courant F_{t-1} ;
2. on calcule, pour chaque exemple, les *pseudo-résidus* correspondant au gradient négatif de la perte par rapport aux prédictions :

$$r_{it} = -\left.\frac{\partial}{\partial F(x_i)} L\bigl(y_i, F(x_i)\bigr)\right|_{F=F_{t-1}} ;$$

   pour une perte quadratique, on obtient simplement $r_{it} = y_i - F_{t-1}(x_i)$;
3. on ajuste un apprenant faible h_t pour approximer ces pseudo-résidus, typiquement par régression aux moindres carrés :

$$h_t \approx \arg\min_h \sum_{i=1}^{n} \bigl(r_{it} - h(x_i)\bigr)^2;$$

4. on choisit un pas optimal β_t (par recherche de ligne) :

$$\beta_t = \arg\min_\beta \sum_{i=1}^{n} L\bigl(y_i, F_{t-1}(x_i) + \beta\, h_t(x_i)\bigr),$$

   puis on met à jour

$$F_t(x) = F_{t-1}(x) + \nu\, \beta_t\, h_t(x).$$

Pour la régression (perte quadratique), cela revient à ajuster successivement les résidus $y_i - F_{t-1}(x_i)$; pour la classification logistique, les pseudo-résidus correspondent à une erreur sur les probabilités estimées. Ainsi, chaque itération corrige systématiquement l'erreur résiduelle, ce qui diminue le biais tout en permettant un contrôle fin de la complexité via ν, la profondeur des arbres et le nombre d'itérations [13].

Des implémentations standards existent dans scikit-learn (`GradientBoostingClassifier`, `GradientBoostingRegressor`) ainsi que dans des bibliothèques dédiées comme XGBoost et LightGBM.

XGBoost et LightGBM : optimisations modernes. XGBoost [5] et LightGBM [21] sont des implémentations très optimisées du gradient boosting sur arbres de décision, qui ajoutent plusieurs idées clés :

— utilisation d'un développement de Taylor d'ordre 2 de la perte pour calculer, pour chaque feuille candidate, le gain associé à une scission en fonction de la somme des gradients G et des hessiennes H ;

— régularisation explicite des arbres via des termes de pénalisation L_2 sur les poids des feuilles et une pénalité sur le nombre de feuilles (paramètres λ et γ) ;

— sous-échantillonnage des observations et des variables (*row/column subsampling*) pour réduire la corrélation entre arbres et accélérer l'entraînement ;

— structures de données basées sur des histogrammes et croissance feuille-par-feuille (LightGBM) pour optimiser l'utilisation de la mémoire et du CPU.

Par exemple, dans XGBoost, le gain d'une scission est approximativement donné par [5]

$$\text{Gain} = \frac{1}{2}\left(\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right) - \gamma,$$

où G_L, H_L (resp. G_R, H_R) sont les sommes des gradients et hessiennes dans les feuilles gauche et droite. Cette expression permet de choisir efficacement les meilleures scissions.

Du point de vue pratique, XGBoost et LightGBM sont devenus des standards de facto pour les problèmes tabulaires de grande dimension, grâce à leur excellent compromis entre biais (faible) et variance contrôlée par une régularisation riche et de nombreux hyperparamètres.

1.4.3 Stacking (stacked generalization)

Le stacking, ou *stacked generalization*, est une méthode d'ensemble à deux niveaux où un modèle dit *méta-modèle* apprend comment combiner au mieux les prédictions de plusieurs modèles de base [35, 29]. Contrairement au vote ou à la simple moyenne, les poids de combinaison sont appris à partir des données, ce qui permet souvent de réduire l'erreur quadratique moyenne (MSE) en régression, ou l'erreur de classification, par rapport à chaque modèle pris isolément [3].

Architecture en couches. On distingue classiquement deux niveaux [35, 29] :

— **Niveau 0 (apprenants de base)** : plusieurs modèles potentiellement très différents (forêt aléatoire, gradient boosting, SVM, k-NN, réseaux neuronaux, etc.) sont entraînés sur les données d'entrée x pour prédire y ;

— **Niveau 1 (méta-modèle)** : un dernier modèle (souvent une régression linéaire pour la régression, ou une régression logistique pour la classification) prend en entrée les prédictions des modèles de niveau 0 et apprend une combinaison optimale.

En notation matricielle pour un problème de régression, si l'on dispose de K modèles de base produisant des prédictions $\hat{y}_k^{(i)}$ pour l'observation i, on forme la matrice de caractéristiques de niveau 1 :

$$Z = \begin{bmatrix} \hat{y}_1^{(1)} & \dots & \hat{y}_K^{(1)} \\ \vdots & & \vdots \\ \hat{y}_1^{(n)} & \dots & \hat{y}_K^{(n)} \end{bmatrix}, \quad y = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix}.$$

Le méta-modèle linéaire cherche alors des coefficients $w \in \mathbb{R}^K$ tels que

$$\hat{y}^{\text{stack}} = Zw$$

approche au mieux y au sens de la MSE. Le choix de w se fait en minimisant

$$\min_w \|y - Zw\|_2^2,$$

éventuellement sous la contrainte $w_k \geq 0$ comme proposé par Breiman [3]. Cette contrainte de non-négativité aide à garantir que le stacking ne dégrade pas les performances par rapport aux meilleurs modèles de base.

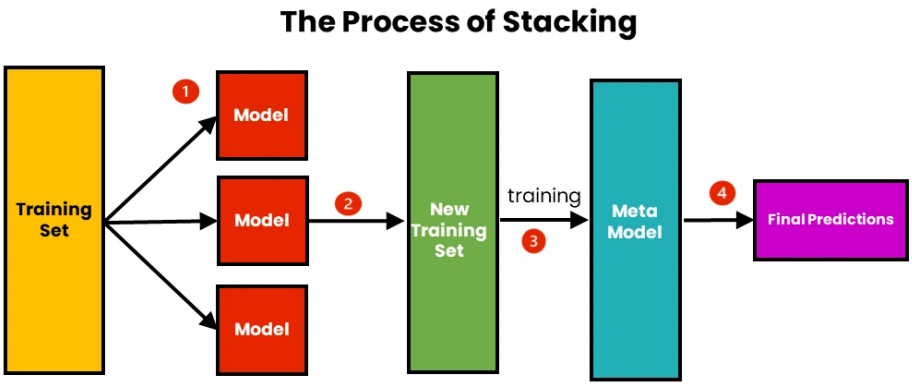

FIGURE 1.6 – Schéma pédagogique du stacking : architecture en couches avec apprenants de niveau 0 et méta-modèle de niveau 1.

Le problème du « data leakage ». Une implémentation naïve du stacking consisterait à entraîner les modèles de base sur tout le jeu d'entraînement D_{train} et à utiliser leurs prédictions sur ces mêmes exemples comme méta-caractéristiques :

$$Z_{ij}^{\text{naïf}} = L_i(x_j), \quad (x_j, y_j) \in D_{\text{train}},$$

où L_i désigne le i-ème modèle de base. Dans ce cas, le méta-modèle est ajusté sur des prédictions issues de données déjà vues par les modèles de niveau 0. On introduit ainsi une fuite d'information (*data leakage*) : les performances apparentes sur l'entraînement sont très optimistes, et le méta-modèle apprend à exploiter des corrélations artificiellement fortes entre les sorties des modèles

de base et la cible [29].

Pour obtenir une estimation non biaisée des sorties des modèles de base, il faut s'assurer que, pour chaque exemple (x_j, y_j), les prédictions utilisées au niveau 1 proviennent de modèles qui n'ont pas été entraînés sur (x_j, y_j). La solution est d'utiliser des prédictions *out-of-fold* (OOF) construites par validation croisée interne.

Etapes algorithmiques (stacking avec validation croisée). La procédure standard de stacking avec prédictions OOF se déroule comme suit :

1. **Sélection des modèles de base.** Le choix des apprenants de niveau 0 est crucial. On recherche :
   - **Diversité** : des modèles capturant des aspects différents du problème (modèles linéaires, arbres, réseaux de neurones, SVM, k-NN, etc.) ;
   - **Complémentarité** : des erreurs faiblement corrélées entre modèles, de sorte que chacun corrige les faiblesses des autres ;
   - **Qualité individuelle** : chaque modèle doit être raisonnablement performant pris isolément (un modèle très mauvais reste difficile à rattraper par le stacking) ;
   - **Hétérogénéité** : on privilégie des familles algorithmiques différentes plutôt que de nombreuses variantes très proches d'un même modèle.
2. **Découpage du jeu d'entraînement.** On partitionne le jeu de données en K plis de validation croisée $F_1, \ldots, F_K$.
3. **Entraînement des modèles de base et génération des méta-caractéristiques.** Pour chaque pli F_k :
   (a) on entraîne chaque modèle de base sur $D \setminus F_k$;
   (b) on applique ces modèles à F_k pour produire des prédictions $\hat{y}_k^{(i)}$ pour les exemples $i \in F_k$.

   En concaténant les prédictions sur tous les plis, on obtient, pour chaque exemple i, un vecteur $(\hat{y}_1^{(i)}, \ldots, \hat{y}_K^{(i)})$ obtenu sur des données *out-of-fold*.
4. **Apprentissage du méta-modèle.** On ajuste ensuite le méta-modèle sur la matrice Z et les vraies cibles y. Le choix du méta-modèle dépend du type de tâche et du compromis biais/variance recherché. Quelques options courantes sont :
   - **Régression linéaire / Ridge** : interprétable, stable, rapide ; bon choix par défaut en régression pour un premier essai ;
   - **Régression logistique** : produit des probabilités bien calibrées ; adaptée à la classification binaire ;
   - **Arbre de décision (profondeur modérée)** : permet de capturer des interactions non linéaires entre les sorties des modèles de base ;
   - **Gradient boosting peu profond** : très performant lorsque l'on dispose de beaucoup de données et que l'on souhaite modéliser des combinaisons complexes de prédicteurs de niveau 0 ;
   - **Réseau de neurones peu profond** : utile pour des schémas de stacking plus profonds ou lorsque les relations entre modèles de base sont fortement non linéaires.

En pratique, on privilégie souvent des méta-modèles simples (régression linéaire ou logistique) pour limiter le risque de surajustement au niveau 1.

5. **Apprentissage final des modèles de base.** On réentraîne enfin tous les modèles de base sur la totalité du jeu d'entraînement, avant de les utiliser en test. En production, la prédiction d'un nouvel exemple $x^\star$ se fait en deux temps : (i) chaque modèle de base produit une prédiction ; (ii) le méta-modèle agrège ces prédictions.

Cette procédure garantit que le méta-modèle a vu des prédictions réalistes (obtenues sur des échantillons non vus par les modèles de base), évitant ainsi un surajustement massif.

Reduction du Biais et la Variance Le stacking agit à deux niveaux :

— **Réduction du biais**le méta-learner apprend à corriger les erreurs systématiques des base learners en apprenant une combinaison non-linéaire (si le méta est un arbre ou un réseau).

— **Réduction de la variance** en utilisant des prédictions croisées, il évite le surajustement et agit comme un régulariseur ; la variance finale est souvent inférieure à celle d'un simple vote ou moyenne.

Mathématiquement, si les erreurs des apprenants de base sont peu corrélées, la variance de la prédiction combinée est proche de

$$\mathrm{Var}\left(\sum_{i=1}^{K} w_i h_i(x)\right) \approx \sum_{i=1}^{K} w_i^2 \,\mathrm{Var}(h_i(x)),$$

où h_i désigne un modèle de base et w_i les poids appris par le méta-modèle. Dans le cas d'un simple stacking avec moyenne ($w_i = 1/K$), on retrouve

$$\mathrm{Var}_{\text{ensemble}} \approx \frac{1}{K} \sum_{i=1}^{K} \mathrm{Var}\big(h_i(x)\big)$$

Limites et risques du stacking. Malgré ses qualités, le stacking présente plusieurs limites qu'il faut garder à l'esprit :

— **Surapprentissage du méta-modèle** : si les lignes de Z sont trop corrélées à y (par exemple en cas de mauvais schéma OOF), le méta-modèle peut mémoriser des patterns propres au jeu d'entraînement plutôt que des relations généralisables.

— **Coût computationnel** : la génération des prédictions OOF nécessite d'entraîner chaque modèle de base sur plusieurs plis (environ $K \times F$ entraînements), puis d'entraîner le méta-modèle ; cela peut devenir coûteux pour de grands ensembles ou des modèles lourds.

— **Complexité de déploiement** : en production, il faut maintenir K modèles de base plus le méta-modèle, ce qui complique la mise à jour, la surveillance et l'observabilité.

— **Interprétabilité réduite** : le système forme une boîte noire à deux niveaux ; il est souvent difficile d'expliquer précisément pourquoi une décision a été prise.

— **Risque de leakage** : toute fuite d'information dans la génération des prédictions OOF (par exemple partage involontaire de données entre plis) invalide les hypothèses théoriques et peut conduire à un méta-modèle fortement biaisé.

1.4.4 Voting (vote majoritaire vs vote probabiliste)

Le vote est la méthode de fusion la plus simple pour les modèles parallèles [6, 15].

Hard voting (vote majoritaire). Chaque modèle donne une étiquette de classe. On choisit la classe qui a reçu le plus de suffrages. Par exemple :

— Modèle A → « chien »

— Modèle B → « chat »

— Modèle C → « chien »

La décision finale est alors « chien » (deux voix contre une).

Soft voting (vote probabiliste). On calcule la moyenne des probabilités prédites pour chaque classe. Cette méthode est plus précise car elle accorde plus d'importance aux modèles « sûrs d'eux ».

Considérons par exemple deux classes (chat, chien) et trois modèles :

— Modèle 1 : probabilités $[P(\text{chien}) = 0{,}4,\ P(\text{chat}) = 0{,}6] \Rightarrow$ prédit « chat »

— Modèle 2 : probabilités $[0{,}4,\ 0{,}6] \Rightarrow$ prédit « chat »

— Modèle 3 : probabilités $[0{,}9,\ 0{,}1] \Rightarrow$ prédit « chien »

En hard voting, la classe « chat » (deux voix) l'emporte. En soft voting, on calcule :

$$\overline{P}(\text{chien}) = \frac{0{,}4 + 0{,}4 + 0{,}9}{3} \approx 0{,}56, \qquad \overline{P}(\text{chat}) = \frac{0{,}6 + 0{,}6 + 0{,}1}{3} \approx 0{,}43.$$

La décision finale est alors « chien » : la forte conviction du modèle 3 compense l'incertitude des deux autres.

1.5 Analyse comparative des approches

Le choix d'une méthode d'ensemble dépend des contraintes du projet (volume de données, temps de réponse, précision requise, robustesse au bruit, coût de calcul, etc.).

Le tableau suivant résume quelques caractéristiques essentielles des trois grandes familles :

Table 1.2 – Comparaison synthétique bagging / boosting / stacking

Critère	Bagging	Boosting	Stacking
Objectif principal	Réduire la variance	Réduire le biais	Maximiser la précis
Type de modèles	Homogènes (arbres profonds)	Homogènes (arbres peu profonds)	Hétérogènes (divers
Entraînement	Parallèle (indépendant)	Séquentiel (dépendant)	Parallèle + méta-a
Sensibilité au bruit	Faible (robuste)	Élevée (risque de sur-ajuster)	Variable
Risque d'overfitting	Faible	Élevé (si non régularisé)	Moyen (si données
Coût de calcul	Efficace (parallélisable)	Lent (entraînement en série)	Très élevé

1.6 Avantages et limites de la combinaison

1.6.1 Avantages

1. **Précision supérieure** : les ensembles dépassent quasi systématiquement les modèles uniques en combinant des perspectives complémentaires.
2. **Généralisation et robustesse** : en lissant les prédictions, les systèmes MCS sont moins sensibles aux anomalies statistiques d'un seul jeu de données.
3. **Traitement des données complexes** : ils permettent de diviser un problème complexe en sous-tâches traitées par des experts dédiés (*divide and conquer*).

1.6.2 Limites

1. **Coût en ressources** : l'entraînement de dizaines ou de centaines de modèles consomme du temps CPU, de la mémoire et de l'énergie.
2. **Opacité (« black box »)** : il est extrêmement difficile d'expliquer pourquoi un ensemble de plusieurs centaines d'arbres de décision a pris une décision spécifique.
3. **Complexité de maintenance** : gérer un pipeline avec plusieurs modèles hétérogènes demande une infrastructure plus lourde qu'un modèle monolithique.

Conclusion

La combinaison de classifieurs représente une approche mature et extrêmement puissante de l'apprentissage automatique. En orchestrant judicieusement la diversité et en s'appuyant sur des fondements statistiques rigoureux, les systèmes multi-classifieurs permettent de transformer une collection de modèles imparfaits en un outil de décision d'une fiabilité exceptionnelle [20, 31, 14, 11].

Conclusion générale

Ce rapport a exploré deux piliers de l'apprentissage automatique contemporain : la **combinaison de classifieurs** et l'**explicabilité des modèles**, ainsi que leurs interactions. Plusieurs enseignements en découlent.

La combinaison de classifieurs (bagging, boosting, stacking, voting) permet d'améliorer la précision et la robustesse des prédictions en s'appuyant sur la diversité et l'agrégation des modèles. Les motivations théoriques (réduction de la variance, du biais ou des deux) et les choix architecturaux (génération de l'ensemble, sélection, fusion) conditionnent directement les performances. En contrepartie, ces systèmes deviennent plus complexes : plusieurs apprenants, mécanismes de pondération ou de méta-apprentissage, et donc une décision plus difficile à décomposer et à justifier.

L'explicabilité apparaît comme une exigence complémentaire à la performance. La distinction entre interprétabilité (structure du modèle lisible) et explicabilité (capacité à rendre compte des décisions) permet de clarifier les objectifs. Les méthodes post-hoc (importance des variables, LIME, SHAP, PDP/ICE) offrent des leviers pour expliquer des modèles a priori opaques, en local ou en global. Leur utilisation dans les systèmes décisionnels est renforcée par les enjeux éthiques et réglementaires (transparence, non-discrimination, droit à l'information).

À l'intersection des deux thèmes, le rapport a mis en évidence le **problème de la boîte noire** propre aux ensembles : une décision collective est plus difficile à attribuer à des facteurs précis qu'une décision d'un modèle unique. Les stratégies d'explicabilité des ensembles (explication au niveau de l'agrégation, au niveau des apprenants de base, ou par approximation) permettent de progresser, mais un **compromis entre performance et compréhension** reste inévitable. Selon le contexte (réglementation, domaine critique, acceptabilité utilisateur), le curseur entre complexité du modèle et niveau d'explicabilité requis devra être ajusté.

En synthèse, la combinaison de classifieurs reste un outil puissant pour la qualité prédictive ; l'explicabilité en est le corollaire indispensable dès que les décisions ont un impact humain ou juridique. Les travaux futurs pourront approfondir l'intégration de contraintes d'explicabilité dès la conception des systèmes d'ensemble et l'évaluation standardisée des méthodes d'explication dans ce cadre.

Bibliographie

[1] Analytics Vidhya. Bagging, boosting and stacking : Ensemble learning in ml models. https://www.analyticsvidhya.com/blog/2023/01/ensemble-learning-methods-bagging-boosting-and-stacking/, 2023. Consulté le 23 février 2026.

[2] Leo Breiman. Bagging predictors. *Machine Learning*, 24(2) :123–140, 1996. Article fondateur du bagging.

[3] Leo Breiman. Stacked regressions. *Technical Report*, 1996. Manuscrit disponible via les archives de Leo Breiman.

[4] Jason Brownlee. Bagging vs boosting vs stacking : Which ensemble method wins in 2025 ? https://machinelearningmastery.com/bagging-vs-boosting-vs-stacking-which-ensemble-method-wins-in-2025/, 2025. Consulté le 23 février 2026.

[5] Tianqi Chen and Carlos Guestrin. Xgboost : A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[6] Codefinity. Learn voting : Hard and soft. https://codefinity.com/courses/v2/3172b303-2b09-492b-8bb9-5bbd1bdcef09/81f3a01b-d537-4478-bc23-3ea1895e4f1f/d66aa7bd-877e-4808-ad40-aa051eabbc4a, 2024. Consulté le 23 février 2026.

[7] William W. Cohen. Ensemble methods for machine learning. https://www.cs.cmu.edu/~wcohen/10-601/ensembles.pdf, 2010. Support de cours, Carnegie Mellon University. Consulté le 23 février 2026.

[8] R. M. O. Cruz, R. Sabourin, and G. D. C. Cavalcanti. Dynamic ensemble selection and instantaneous pruning for regression. In *ESANN 2014*, 2014. Consulté le 23 février 2026.

[9] R. M. O. Cruz, R. Sabourin, and G. D. C. Cavalcanti. Deslib : A dynamic ensemble selection library in python. *Journal of Machine Learning Research*, 21(8) :1–5, 2020. Consulté le 23 février 2026.

[10] DESlib project. Welcome to deslib documentation ! https://deslib.readthedocs.io/, 2020. Consulté le 23 février 2026.

[11] Luis Fred. Multiple classifier systems — a brief introduction. https://medium.com/luisfredgs/multiple-classifier-systems-a-brief-introduction-71238d9c794f, 2023. Consulté le 23 février 2026.

[12] Yoav Freund and Robert E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1) :119–139, 1997.

[13] Jerome H. Friedman. Greedy function approximation : A gradient boosting machine. *Annals of Statistics*, 29(5) :1189–1232, 2001.

[14] GeeksforGeeks. A comprehensive guide to ensemble learning. `https://www.geeksforgeeks.org/machine-learning/a-comprehensive-guide-to-ensemble-learning/`, 2023. Consulté le 23 février 2026.

[15] GeeksforGeeks. Voting classifier. `https://www.geeksforgeeks.org/machine-learning/voting-classifier/`, 2023. Consulté le 23 février 2026.

[16] Vibhav Gogate. Bias/variance tradeoff and ensemble methods. `https://personal.utdallas.edu/~vibhav.gogate/ml/2020f/lectures/EnsembleMethods.pdf`, 2020. Support de cours, Université du Texas à Dallas. Consulté le 23 février 2026.

[17] Great Learning. Understanding the ensemble method bagging and boosting. `https://www.mygreatlearning.com/blog/bagging-boosting/`, 2023. Consulté le 23 février 2026.

[18] Laurent Heutte. Combinaison de classifieurs. `http://lheutte.free.fr/rdf/ArchiCombi.pdf`, 2003. Consulté le 23 février 2026.

[19] IBM. What is bagging ? `https://www.ibm.com/think/topics/bagging`, 2024. Consulté le 23 février 2026.

[20] IBM. What is ensemble learning ? `https://www.ibm.com/think/topics/ensemble-learning`, 2024. Consulté le 23 février 2026.

[21] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm : A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*, pages 3146–3154, 2017.

[22] Albert Ko, Robert Sabourin, and Alceu de Souza Britto Jr. From dynamic classifier selection to dynamic ensemble selection. In *Proceedings of the 7th International Conference on Machine Learning and Applications*, 2007. Consulté le 23 février 2026.

[23] Anders Krogh and Jesper Vedelsby. Neural network ensembles, cross validation, and active learning. In *Advances in Neural Information Processing Systems 7 (NIPS 1994)*, pages 231–238, 1995. Article fondateur de l'identité de Krogh et Vedelsby sur l'ambiguïté des ensembles.

[24] Ludmila I. Kuncheva and Juan J. Rodríguez. A unified theory of diversity in ensemble learning. *Information Fusion*, 2023. Résumé disponible sur ResearchGate, consulté le 23 février 2026.

[25] G. Metzler. Machine learning td – méthodes ensemblistes. `https://eric.univ-lyon2.fr/gmetzler/courses/contents/machinelearning/td4.pdf`, 2022. Travaux dirigés, Université Lyon 2. Consulté le 23 février 2026.

[26] NS Academy. What is ensemble learning (bagging vs boosting vs stacking) ? `https://medium.com/@sharetonschool/what-is-ensemble-learning-bagging-vs-boosting-vs-stacking-c3c8cc94a3d5`, 2023. Consulté le 23 février 2026.

[27] Robert E. Schapire. A brief introduction to boosting. *IJCAI*, 99 :1401–1406, 1999.

[28] scikit-learn-contrib. Deslib : A python library for dynamic classifier and ensemble selection. `https://github.com/scikit-learn-contrib/DESlib`, 2020. Consulté le 23 février 2026.

[29] Kai Ming Ting and Ian H. Witten. Issues in stacked generalization. *Journal of Artificial Intelligence Research*, 10 :271–289, 1999.

[30] Turing College. Ensemble learning : Key techniques explained. `https://www.turingcollege.com/blog/ensemble-learning-2`, 2024. Consulté le 23 février 2026.

[31] Ultralytics. Exploration de l'apprentissage d'ensemble : son rôle dans l'ia et la ml. `https://www.ultralytics.com/fr/blog/exploring-ensemble-learning-and-its-role-in-ai-and-ml`, 2024. Consulté le 23 février 2026.

[32] Wikipedia. Bias–variance tradeoff. `https://en.wikipedia.org/wiki/Bias%E2%80%93variance_tradeoff`, 2026. Consulté le 23 février 2026.

[33] Wikipedia. Ensemble learning. `https://en.wikipedia.org/wiki/Ensemble_learning`, 2026. Consulté le 23 février 2026.

[34] Wikipédia. Théorème du jury. `https://fr.wikipedia.org/wiki/Th%C3%A9or%C3%A8me_du_jury`, 2026. Consulté le 23 février 2026.

[35] David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2) :241–259, 1992.

[36] Kaleemullah Younas. Bagging, boosting and stacking : Ensemble learning in ml models. `https://medium.com/@kaleemullahyounas123/bagging-boosting-and-stacking-ensemble-learning-in-ml-models-8eb2f73afb07`, 2023. Consulté le 23 février 2026.